

# Advanced Java

## Servlet :-

### What is Servlet ?

- A Servlet is a Java program that runs on a web server.
- It is used to create dynamic web content.
- It acts as a middle layer between the client request and the server.
- It connects the browser with server-side resources like databases or applications.

### Why Use Servlet ?

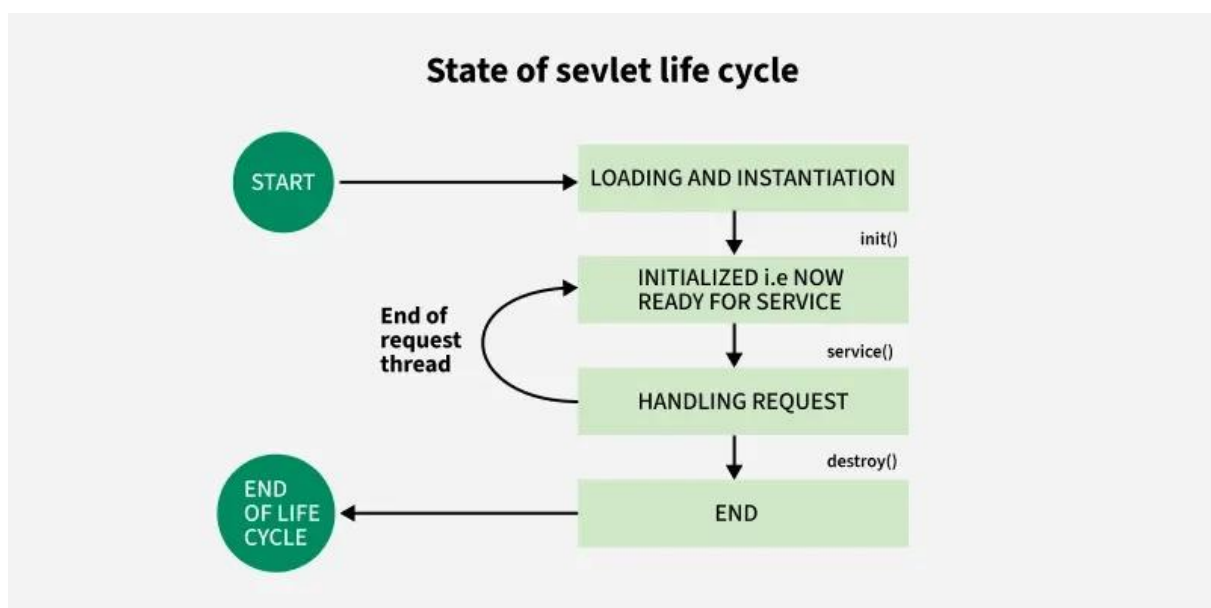
- To create dynamic web pages using Java.
- To handle client requests and server responses efficiently.
- To build secure, robust, and platform-independent web applications.
- To improve performance compared to traditional CGI (Common Gateway Interface).
- To easily connect with databases and other backend systems.
- To take advantage of reusability and scalability in web development.

### How it Works ?

- The client (usually a web browser) sends a request to the web server.
- The web server passes the request to the Servlet container (like Tomcat).
- The Servlet container loads the required Servlet class.
- The Servlet processes the request (e.g., reads data, interacts with a database).
- The Servlet generates a response (usually HTML).
- The web server sends the response back to the client browser.

### Life Cycle of Servlet :-

The Servlet life cycle consists of four main stages:



## 1. Loading and Instantiation

- The servlet class is loaded into memory by the container.
- Then, an instance of the servlet is created.

## 2. Initialization (`init()` method)

- The container calls the `init()` method once.
- It is used to initialize the servlet before handling requests.

## 3. Request Handling (`service()` method)

- The `service()` method is called for every client request.
- It processes the request and sends back a response.

## 4. Destruction (`destroy()` method)

- The `destroy()` method is called once before the servlet is unloaded.
- It is used to release resources or perform cleanup.

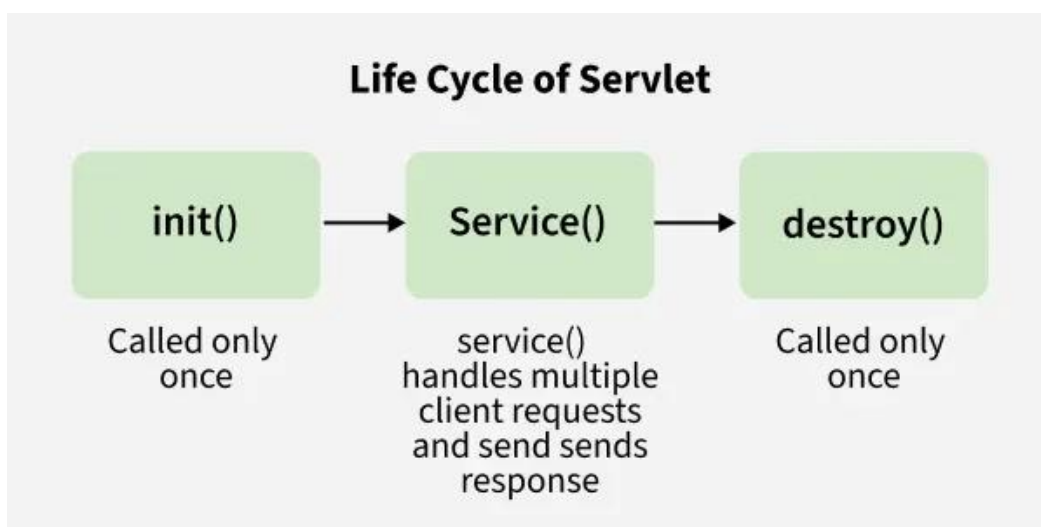
## 5. Garbage Collection

- After destruction, the servlet object is no longer needed.
- It becomes eligible for garbage collection by the JVM.

## Servlet Life Cycle Methods :-

There are three life cycle methods of a Servlet:

1. `init()`
2. `service()`
3. `destroy()`



### **1. init() :-**

- This method is called only once when the servlet is first loaded.
- It is used for initializing resources like database connections or configuration settings.

### **2. service() :-**

- This method is called each time a client request is received.
- It is responsible for processing requests and generating responses.

### **3. destroy() :-**

- This method is called once when the servlet is being removed from memory.
- It is used to release resources and perform cleanup tasks.

## Java Server Page ( JSP ) :-

### What is JSP ?

- JSP is a server-side technology used to create dynamic web content.
- It allows Java code to be embedded directly into HTML pages.
- JSP pages are executed on the server to generate dynamic responses for clients.
- It is an extension of the Servlet technology, making web development easier and faster.

### Why Use JSP ?

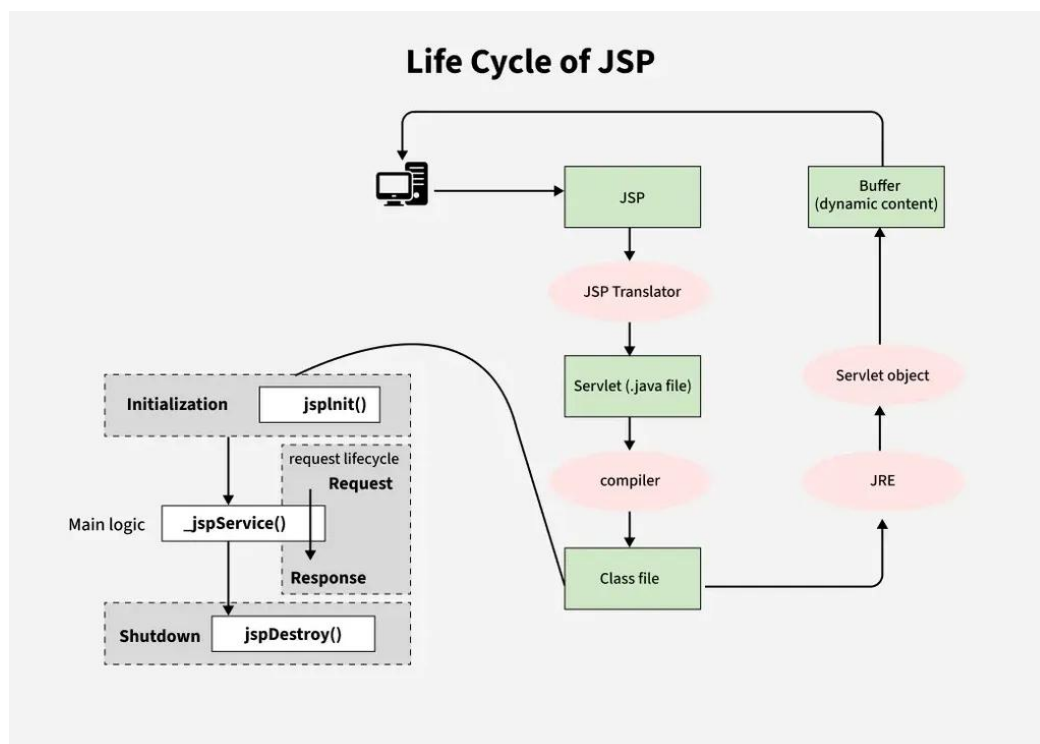
- To create dynamic web pages easily.
- Simpler and faster than servlets.
- Separates design from business logic.
- Supports reusable components.
- Automatically compiled and platform-independent.

### How it Works ?

- The client sends a request for a .jsp page.
- The web server passes the request to the JSP engine.
- The JSP page is converted into a Servlet by the container.
- The Servlet is then compiled and executed to generate HTML output.
- The response is sent back to the client browser.

### Life Cycle of JSP :-

The JSP life cycle consists of five main phases:



- The life cycle of JSP includes several phases from creation to destruction.
- The JSP page is first created and then translated into a servlet.
- The servlet lifecycle manages its execution and response handling.
- The entire process is handled automatically by the JSP engine.

#### **1. Translation Phase**

- The JSP file is translated into a Java Servlet by the JSP container.
- This happens only once when the page is first requested.

#### **2. Compilation Phase**

- The translated servlet file is compiled into bytecode.
- This bytecode is then loaded into memory for execution.

#### **3. Initialization (jspInit() method)**

- The container calls the jspInit() method once to initialize the JSP.
- It is used to set up resources needed for processing requests.

#### **4. Request Processing (\_jspService() method)**

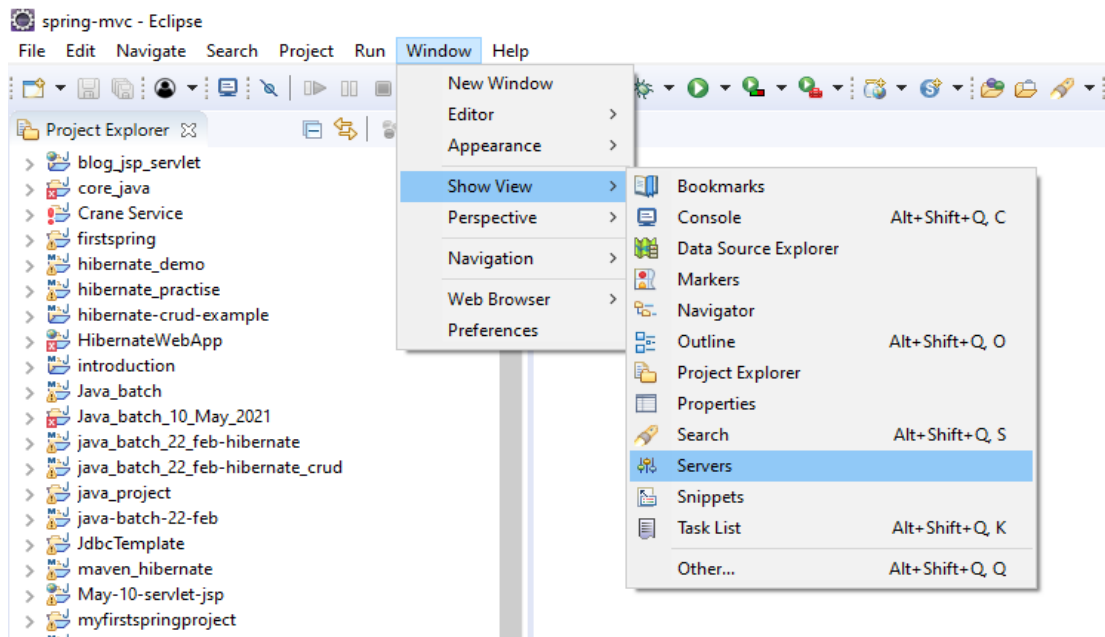
- The \_jspService() method is called for each client request.
- It handles the request and generates a dynamic response.

#### **5. Destruction (jspDestroy() method)**

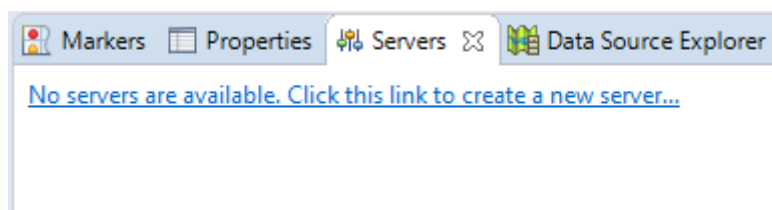
- The jspDestroy() method is called when the JSP is about to be removed.
- It is used to release resources and perform cleanup tasks.

## How to add tomcat Server ?

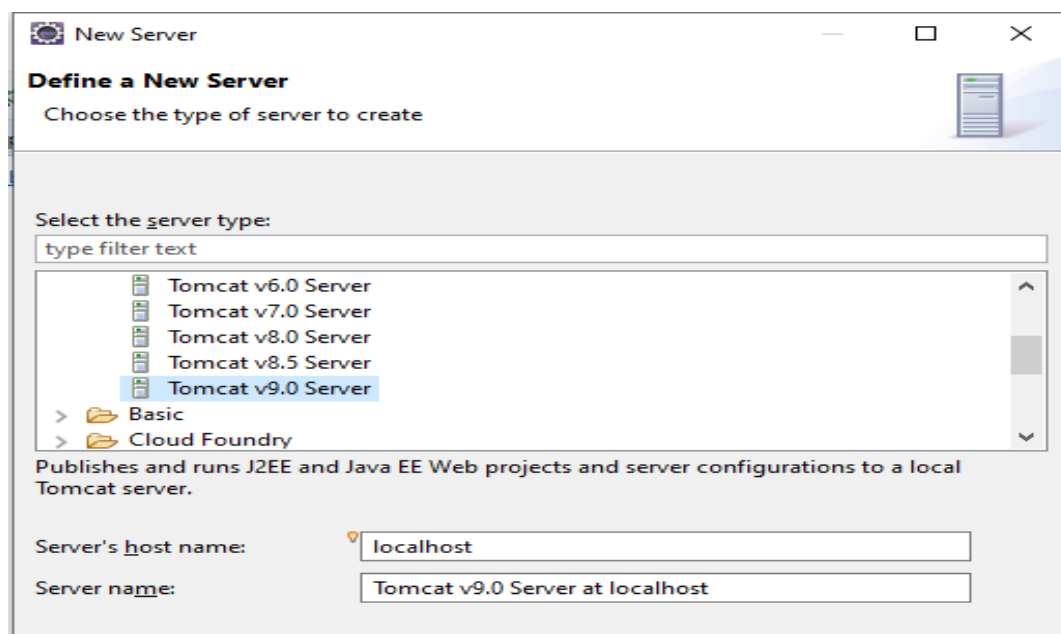
### 1. Step 1



### 2. Step 2

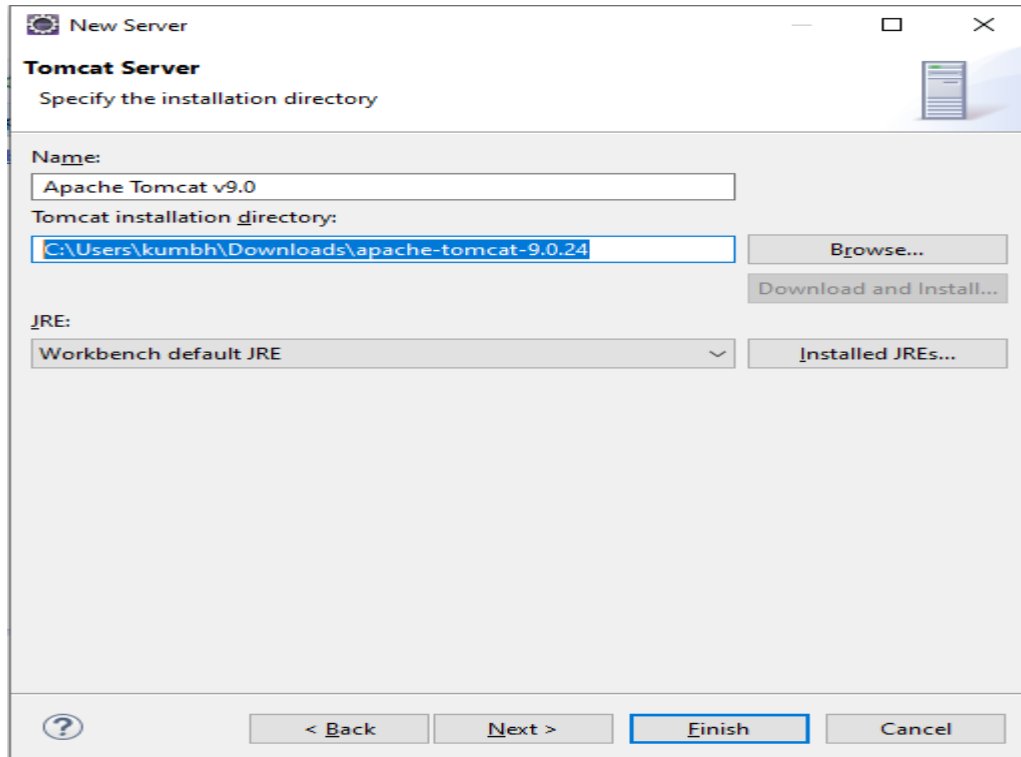


### 3. Open Apache folder- check & download your tomcat version zip file & extract it



4. Click Next

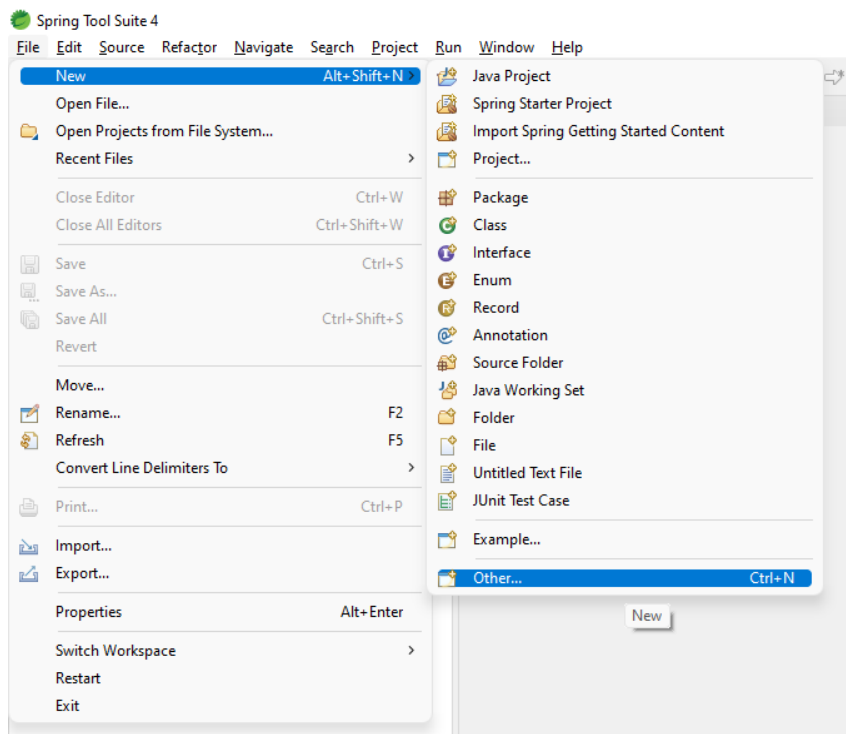
5. Browse tomcat folder from your folder



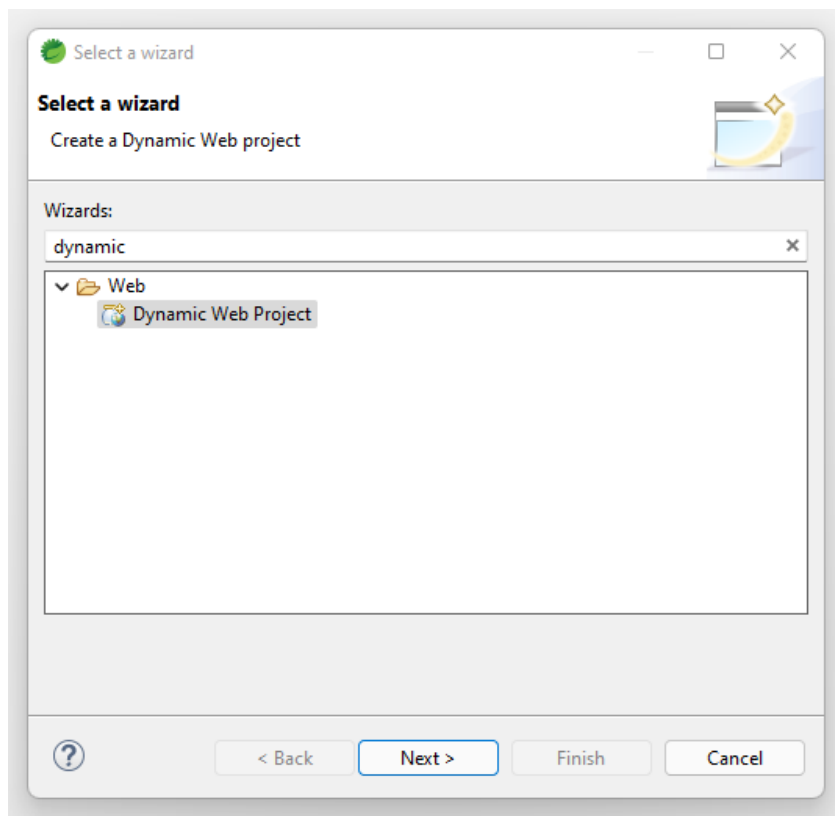
6. Finish

## How to Create a Dynamic Web Project in Eclipse IDE ?

Step 1: Open your Eclipse IDE then go to the File > New > Other as shown in the below image.



Step 2: Now in the select wizard search for the Dynamic Web Project as shown in the below image. And click on the Next button.



Step 3: In the next screen you have to provide your project name as per your choice and don't touch anything else for now. Click on the Next button.

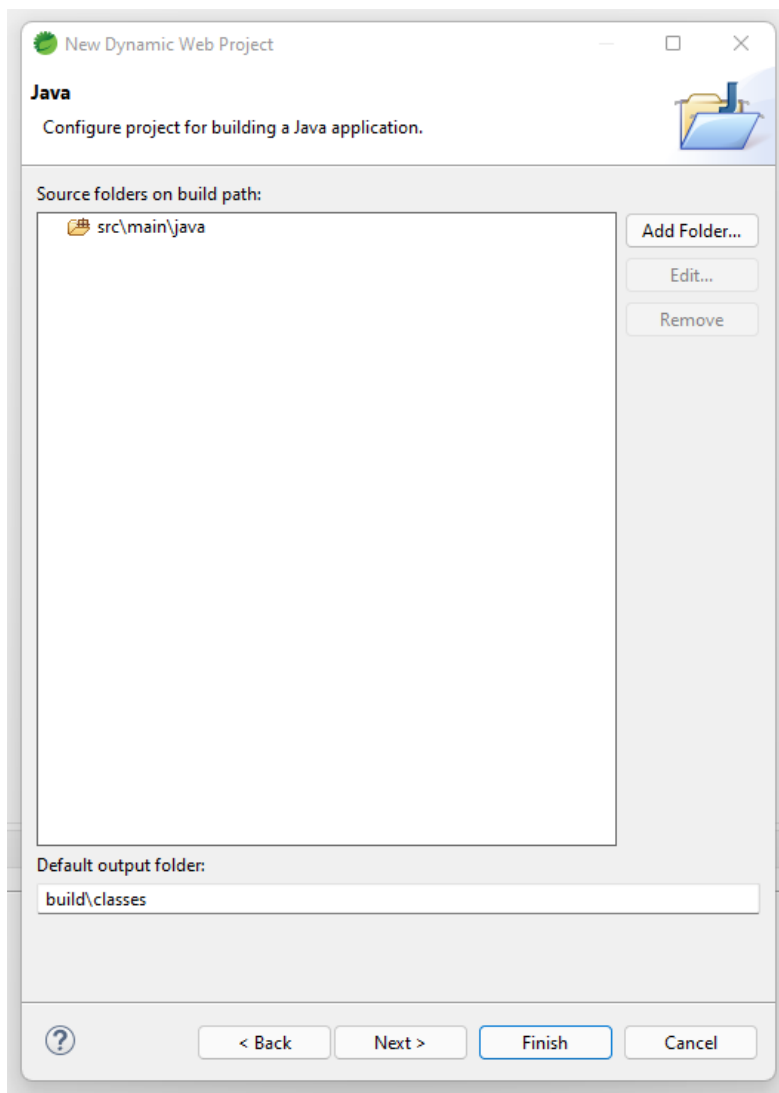
The screenshot shows the 'New Dynamic Web Project' wizard in the Eclipse IDE. The window title is 'New Dynamic Web Project'. The main heading is 'Dynamic Web Project' with a subtitle: 'Create a standalone Java-based Web Application or add it to a new or existing Enterprise Application.' There is a small icon of a jar and a globe.

The wizard is divided into several sections:

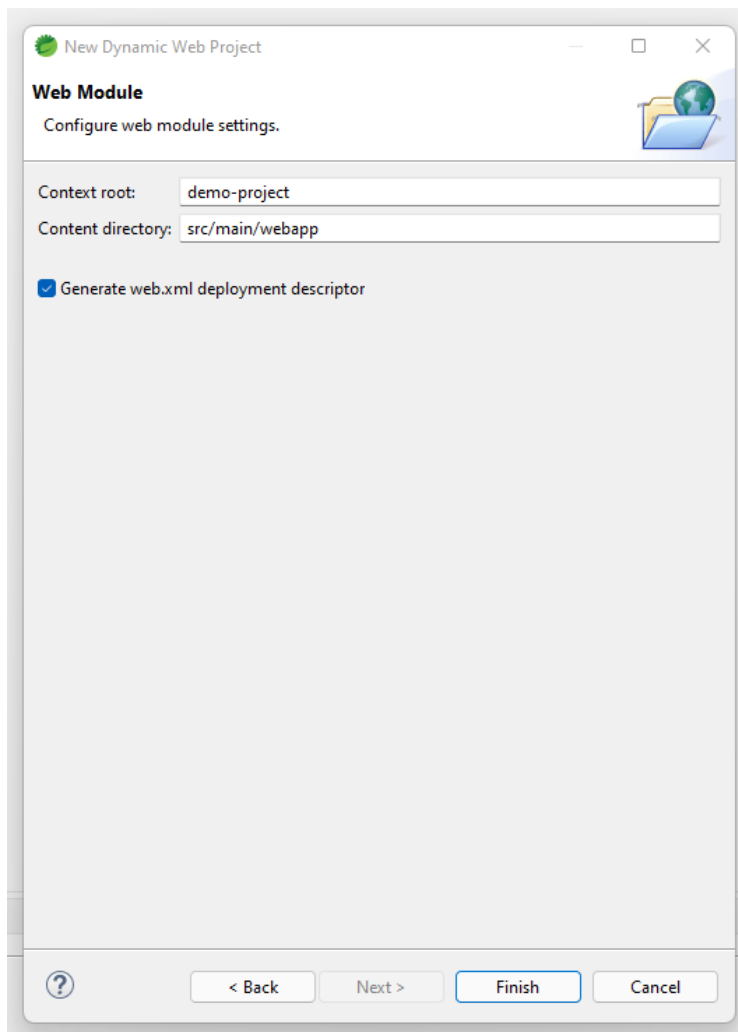
- Project name:** A text field containing 'demo-project'.
- Project location:** A section with a checked checkbox 'Use default location'. Below it, a 'Location:' text field shows 'D:\demo-project' and a 'Browse...' button.
- Target runtime:** A dropdown menu showing 'Apache Tomcat v9.0' and a 'New Runtime...' button.
- Dynamic web module version:** A dropdown menu showing '4.0'.
- Configuration:** A dropdown menu showing 'Default Configuration for Apache Tomcat v9.0' and a 'Modify...' button. Below this, a note states: 'A good starting point for working with Apache Tomcat v9.0 runtime. Additional facets can later be installed to add new functionality to the project.'
- EAR membership:** A section with an unchecked checkbox 'Add project to an EAR'. Below it, an 'EAR project name:' text field shows 'EAR' and a 'New Project...' button.
- Working sets:** A section with an unchecked checkbox 'Add project to working sets' and a 'New...' button. Below it, a 'Working sets:' text field is empty and a 'Select...' button is present.

At the bottom of the wizard, there is a navigation bar with four buttons: '< Back', 'Next >', 'Finish' (which is highlighted with a blue border), and 'Cancel'. A help icon (?) is also present on the left.

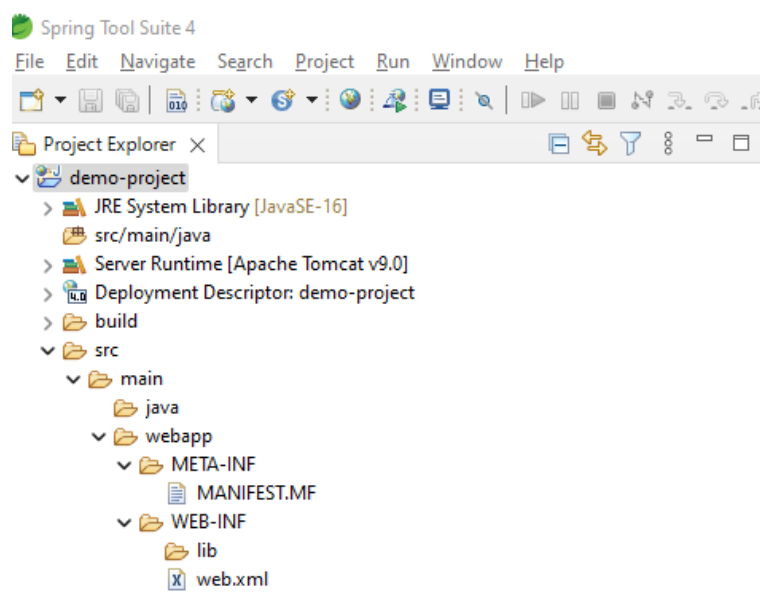
Step 4: In the next screen just click on the Next button.



Step 5: In the last screen just check out the Generate web.xml deployment descriptor box because we need it during the development of a Spring MVC project. Now click on the Finish button and you are done.

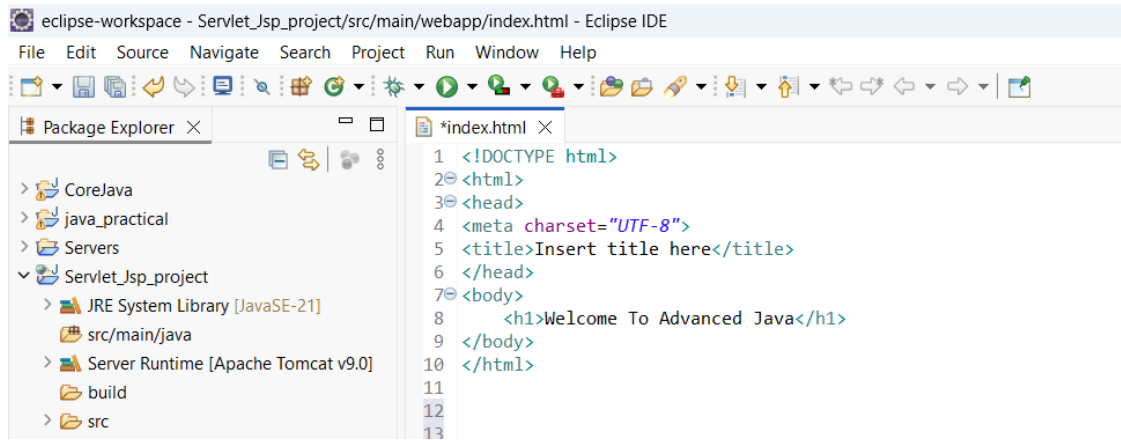


Now your Dynamic Web Project is ready and below is the file structure.



## Implementation of JSP\_Servlet :-

### Example :-



### Output :-

